FARMPULSE AI · GRADUATION PROJECT ARCHITECTURE

Three-Agent Architecture

Computer Vision, Analyst (ML) and Reinforcement Learning agents, the models inside each, and a 6-month plan for a team of four

VERSION	FOLLOWS	TEAM · TIME
1.0 · 24 October 2026	Project README (vision-only stage)	4 students · 6 months · part-time

What it is

The README pipeline (video analyzer, records and dashboard, ML suspicion classifier, RL prioritisation and alerts) organised into three AI agents plus a shared records store, a dashboard and an alert dispatcher.

How many models

11 models in total. 6 need training or fine-tuning (detector, cow ID, posture, mouth-open, suspicion classifier, PPO policy). The other 5 are pretrained or rule-based, so they work without new labels.

Sized for a graduation project

Runs on one GPU PC or laptop plus free Colab or Kaggle GPUs. It processes recorded camera clips in batches, uses one database, and has no cloud, Kubernetes or live multi-camera streaming.

Fits the time you have

About 10 hours a week per student alongside courses, roughly 1,000 person-hours in total. Work is split into must-have and should-have, and two exam buffers are built into the plan.

1 Goal and scope
2 High-level architecture
3 Agent 1 · Computer Vision
4 Agent 2 · Analyst (ML)
5 Agent 3 · Reinforcement Learning
6 Shared parts
7 Technology and hardware
8 Team and 6-month plan
9 Risks and what to cut
10 Evaluation
11 Model summary

1 Goal and scope

Following the README, FarmPulse AI analyses camera footage, links each observation to a cow and a time, and saves a record per cow. It uses those records to separate cows that look normal from cows that may need attention, ranks suspicious cows by priority and an estimated time window, and alerts the owner or vet through the dashboard, SMS and WhatsApp. Possible intruders get a separate, immediate security alert. Every output supports a human decision and is not a diagnosis.

> **What "agent" means here.** Each agent is an independent Python package with one job, a defined input and a defined output. It runs as its own step in the pipeline, so one student can own it end to end. These are not LLM agents.

In scope (from the README)

- Video analysis: cow detection, tracking, identity, behaviour (eating, drinking, lying, walking, standing), visible state (mouth open) and intruder detection.
- A per-cow record for each observation interval, plus a dashboard and a weekly report.
- A suspicious or not-suspicious flag with an estimated time window and confidence.
- RL prioritisation: priority, urgency and a suggested action.
- Alerts on the dashboard, by SMS and by WhatsApp, and owner or vet feedback.

Simplified or left out (graduation scope)

- One farm and 1–3 fixed cameras. Clips are processed in batches (for example every 15 minutes) instead of live streaming.
- No audio (README future work), no cloud, no multi-farm support, no marketplace.
- The RL policy is trained in a simulator and evaluated offline. It is never allowed to act on its own.
- The dashboard is a working prototype, not production quality (README).

2 High-level architecture

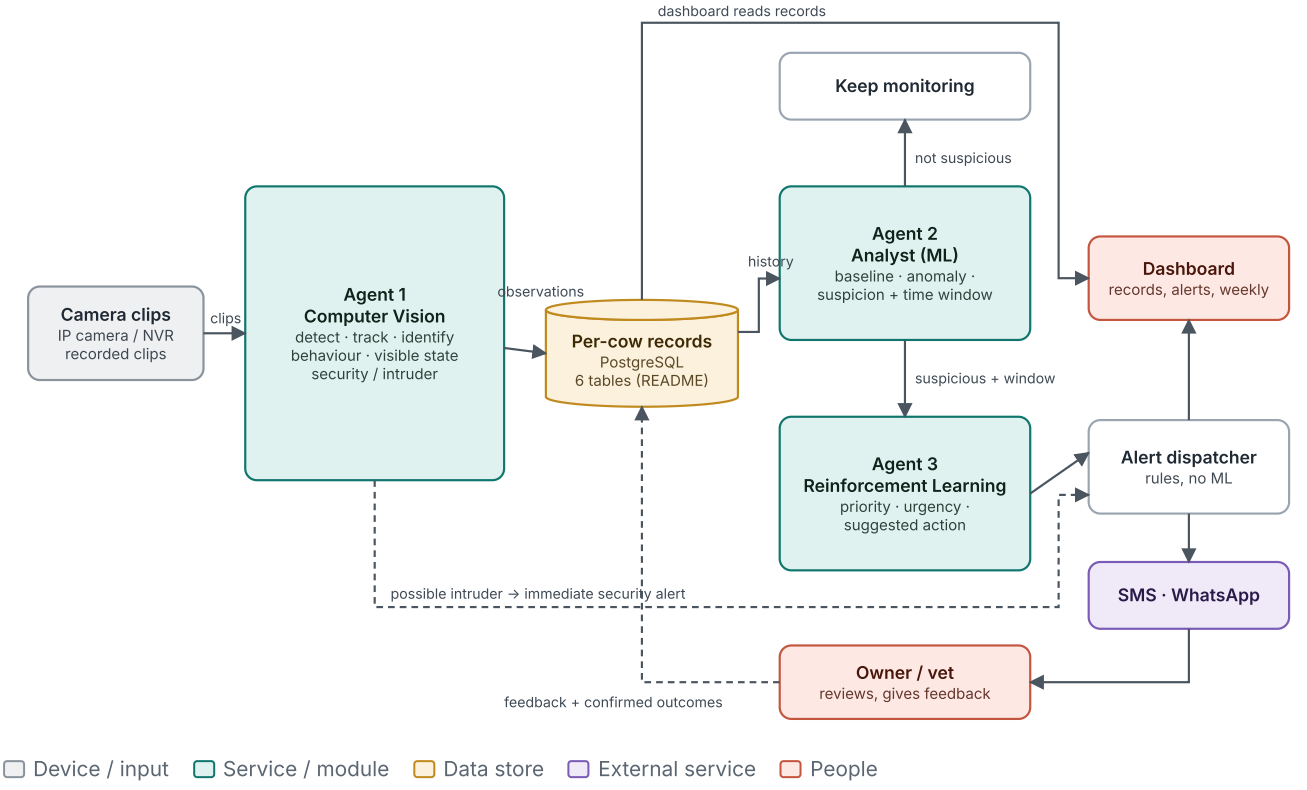


Figure 1. The three agents and the shared parts. Green boxes are AI agents; white boxes are ordinary software with no ML.

#	Step	What happens	Owner
1	Capture	Camera clips tagged with camera, zone and time are saved to a folder (from the camera's NVR or a recording script).	Shared script
2	Agent 1 · Computer Vision	Detects, tracks and identifies cows and labels behaviour and visible state. Possible intruders go straight to the alert dispatcher.	Students 1 and 2
3	Records	One row per cow per 5-minute interval. Cows that cannot be identified are kept as unassigned.	Student 3
4	Agent 2 · Analyst (ML)	Reads current and past records and outputs suspicious or not, with an estimated time window and confidence.	Student 3
5	Agent 3 · RL	For suspicious cows, chooses priority, urgency and a suggested action from an approved list.	Student 4
6	Alert dispatcher	Sends to the dashboard, SMS and WhatsApp according to priority, and avoids duplicate alerts.	Student 4
7	Feedback	The owner or vet confirms or rejects the alert and records the outcome. This is stored separately and used to evaluate the models.	Student 3 (UI)

As the README requires, observations, predictions, alerts and confirmed outcomes are stored in separate tables. That keeps clear what was measured, what was predicted and what was confirmed.

3 Agent 1 · Computer Vision

Input: a video clip with camera, zone and time. **Output:** observations (cow ID or *unassigned*, behaviour seconds, visible state, confidence) and security events. This agent covers the README's CV-M1 (behaviour), CV-M2 (visible state) and CV-M3 (security). It builds on the code already in `cow-detection` and `cow_hide_identification`.

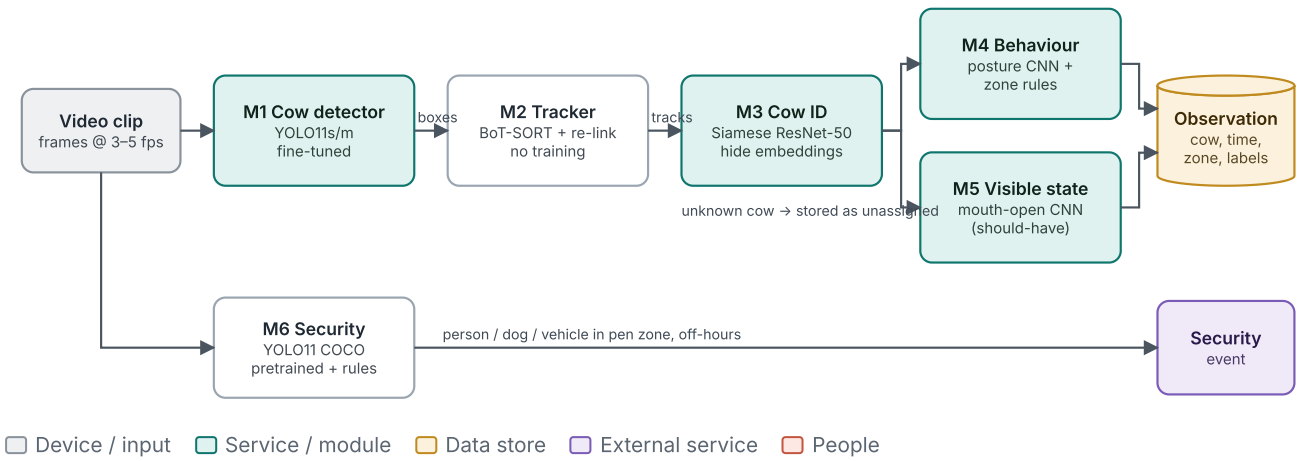


Figure 2. Models inside the Computer Vision agent. Green boxes are trained or fine-tuned; white boxes are pretrained or rule-based.

Model	Task	Approach	Data	Priority
M1 Cow detector	Find every cow in each frame	YOLO11s or YOLO11m (already used), fine-tuned on about 1,500 labelled farm frames, including crowded feeding and night footage where the COCO model misses cows today.	Farm frames, labelled in CVAT or Roboflow	Must
M2 Tracker	Keep the same number on a cow across frames	BoT-SORT with the existing cattle settings and `relink.py`. No training, only tuning.	None	Must
M3 Cow ID	Turn a track into a real cow ID	The Siamese ResNet-50 already written in `cowid` (128-d embeddings, triplet loss) matched against a gallery of registered cows. Below a threshold the track is stored as unassigned. Solid-black cows are a known weak point.	Crops of 15–30 cows, 20+ per cow, from several days	Must
M4 Behaviour	Eating, drinking, lying, walking, standing	A posture classifier (YOLO11s-cls lying or standing, script in `train_posture.py`) plus the zone and motion rules in `behaviour.py`.	Posture crops (1,500 already cut), more night footage	Must
M5 Visible state	Detect mouth open (README example)	A small classifier on head crops (YOLO11n-cls or MobileNetV3). Kept as should-have because positive examples are rare.	Head crops, a few hundred labelled	Should
M6 Security	Possible intruder	Pretrained YOLO11 COCO classes (person, dog, vehicle) with a rule for pen zones and off-hours. No training.	None	Must

```
Observation output (one per cow per 5-minute interval)
{"cow_id": "C104", "observed_at": "2026-03-02T18:00:00Z", "zone": "feeding-area", "camera_id": "camera-
03",
 "vision": {"behaviour_s": {"eating": 212, "lying": 0, "standing": 61, "walking": 12, "drinking": 15},
            "state": ["mouth_open"], "confidence": 0.88, "id_score": 0.84},
 "source_refs": ["camera-03/clip-001"]}
```

4 Agent 2 · Analyst (ML)

Input: per-cow records and their history. **Output:** suspicious or not, an estimated time window (24, 48 or 72 hours) and a confidence score with the top reasons. Following the README, it starts with transparent rules and adds a supervised classifier only when there are enough labelled examples.

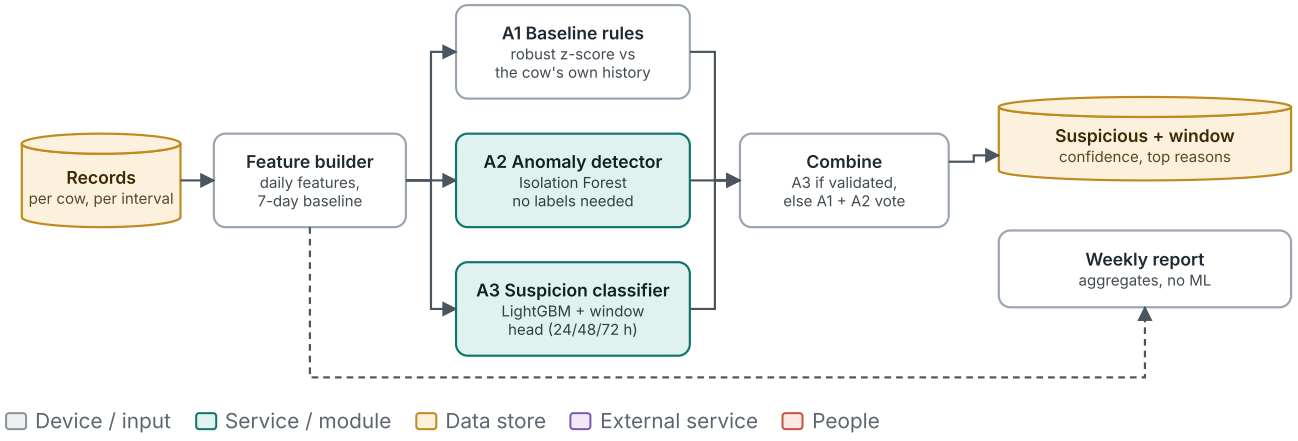


Figure 3. Models inside the Analyst agent. All three run every time, so you can compare them directly in the thesis.

> **Be honest about labels.** Six months on one farm will probably produce only a few vet-confirmed illnesses. Plan to present A1 and A2 as the working system and A3 as a demonstrated method. Report its results with their sample size, as the README says (no claims before held-out evaluation).

Model	Approach	Labels needed	Priority
Feature builder	For each cow per day: feeding minutes, drinking visits, lying minutes and bouts, walking minutes, mouth-open events and data coverage. Also the change from the cow's own 7-day baseline.	None	Must
A1 Baseline rules	Robust z-score (median and MAD) against the cow's own history. For example, flag when feeding drops by more than 3 deviations while lying rises, sustained over two 6-hour windows. Thresholds are agreed with the vet. This is the README's "transparent rules" starting point.	None	Must
A2 Anomaly detector	Isolation Forest on the daily feature vectors, trained on normal days. It catches unusual combinations that single rules miss.	None (unsupervised)	Must
A3 Suspicion classifier	LightGBM on windowed features. A second head predicts the time window as one of 3 classes (24, 48 or 72 h). It is trained on vet-confirmed cases plus vet-reviewed rule flags, and on injected anomalies if real cases are too few.	Vet feedback; at least 30–50 positive cases to report honestly	Should

Combine	If A3 passes evaluation, use A3. Otherwise flag a cow when A1 fires or when both A1 and A2 agree. The output says which models fired.	None	Must
Weekly report	A 7-day summary per cow and for the herd. Plain aggregation with no ML.	None	Must

5 Agent 3 · Reinforcement Learning

Input: suspicious cows from the Analyst plus each cow's history. **Output:** priority (low, medium or high), urgency and a suggested action. As in the README, the time estimate comes from the Analyst and the RL agent decides *what to do and how urgently*.

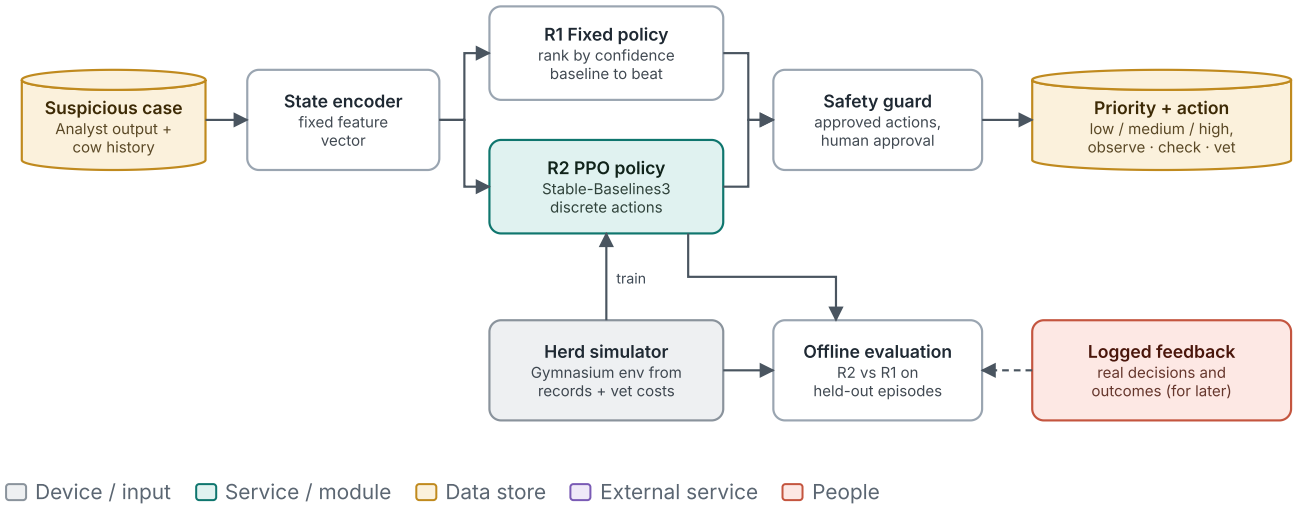


Figure 4. Models inside the RL agent. PPO learns in a simulator and is compared offline against a fixed rule. A person approves every real action.

RL formulation

Element	Definition
State	Analyst confidence and time window, how many models fired, days since the last alert for this cow, recent trend in feeding and lying, data coverage, number of other open alerts, and time of day.
Actions (4)	Monitor (low) · re-check in 2 h (medium) · request a farm check today (high) · call the vet now (urgent). These are the README's approved options only.
Reward	Positive for catching a true case early. Large penalty for a missed case. Smaller penalties for false alarms, late alerts and unnecessary vet calls. The cost values are agreed with the vet.
Environment	A Gymnasium simulator. Cows follow their real recorded baselines, and illness episodes are injected with realistic patterns (feeding falls, lying rises over 1–3 days). Each episode is one simulated week of the herd.

Model	Approach	Priority
R1 Fixed policy	Rank by Analyst confidence and time window and map to actions with thresholds. This is the README's simple fixed rule and the baseline the RL policy must beat.	Must
R2 PPO policy	Stable-Baselines3 PPO with a small MLP, trained in the simulator (CPU is enough). DQN is a quick comparison if time allows.	Must
Safety guard	Only approved actions are allowed. Urgent actions always need a person to confirm. The policy never trains online from unverified alerts (README).	Must
Offline evaluation	Compare R2 with R1 on held-out simulated episodes (cases caught, lead time, false alarms, vet calls). Logged real decisions are kept for future work.	Must

6 Shared parts (no ML)

Part	What it does	How
Records store	The README's six tables: `cow_profiles`, `cow_observations`, `predictions`, `alerts`, `feedback_outcomes`, `weekly_reports`.	PostgreSQL (or SQLite to start), SQLAlchemy
Pipeline runner	Runs the agents in order on each new batch of clips. Each agent reads from and writes to the database, so each can be tested on its own.	Python script with APScheduler (every 15 min)
Dashboard	Cow list and status, cow timeline, alert inbox with confirm and reject, weekly report, and video playback with boxes (already built).	Extend the existing FastAPI `app.py` and `static/`
Alert dispatcher	Channel by priority: urgent and high go to dashboard, WhatsApp and SMS; medium to dashboard and WhatsApp; low to dashboard only. Avoids duplicates and records delivery and acknowledgement. Security alerts go out immediately.	Twilio trial or sandbox (SMS and WhatsApp)

7 Technology and hardware

Need	Choice
Language	Python 3.11 throughout (the team already uses it)
Computer vision	PyTorch, Ultralytics YOLO11, BoT-SORT, OpenCV, the existing `cowid` package
Analyst ML	pandas, scikit-learn (Isolation Forest), LightGBM, SHAP for top reasons
RL	Gymnasium (simulator), Stable-Baselines3 (PPO, DQN)
Backend and UI	FastAPI and the existing HTML/JS dashboard, Chart.js
Data and labelling	PostgreSQL, CVAT or Roboflow for labels, Git and GitHub, DVC (optional)
Alerts	Twilio trial account (SMS plus WhatsApp sandbox)
Hardware	1–3 fixed IP cameras at the farm (or the existing CCTV), one PC with an NVIDIA GPU or a laptop, and free Google Colab or Kaggle GPUs for training

8 Team and 6-month plan

The plan assumes about **10 hours a week per student** during term, so about 260 hours each and 1,040 in total. Each student owns one area end to end. Data collection at the start and integration at the end are done together.

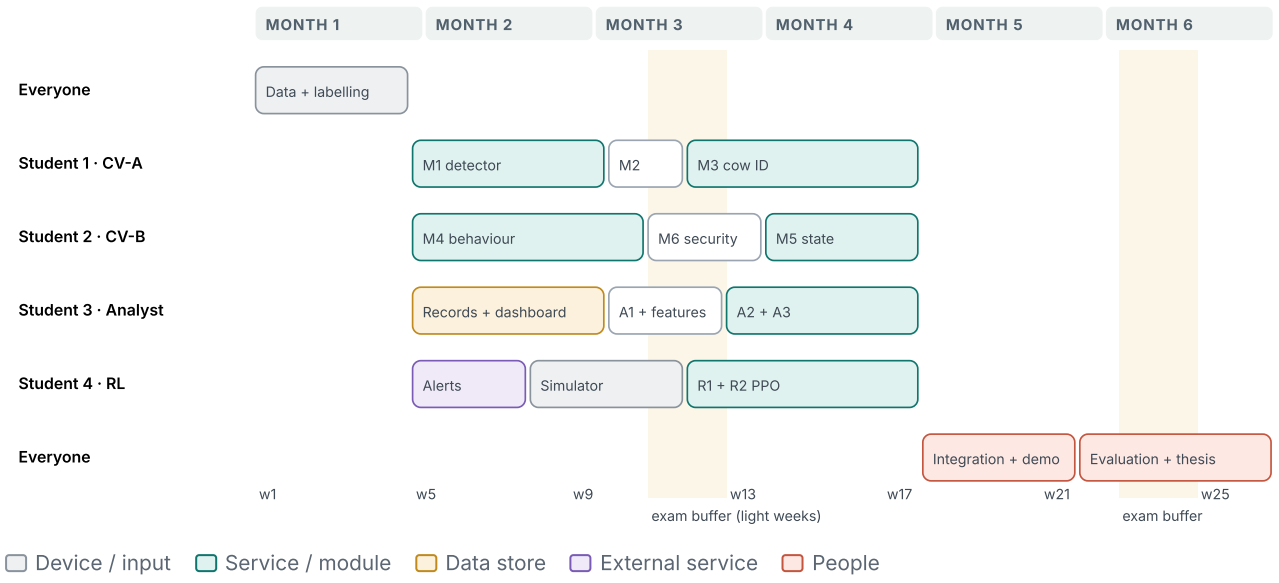


Figure 5. Six-month plan (26 weeks). Shaded weeks are lighter-load buffers for midterms and finals; move them to match your university calendar.

Student	Owns	Delivers
Student 1 · CV-A	M1 detector, M2 tracker, M3 cow ID, the cow gallery	Detector and ID models with metrics; ID chapter
Student 2 · CV-B	M4 behaviour, M5 visible state, M6 security, zones	Behaviour, state and security outputs with metrics
Student 3 · Analyst	Records database, dashboard, feature builder, A1, A2, A3, weekly report	Suspicion results and the dashboard
Student 4 · RL	Simulator, R1, R2, safety guard, alert dispatcher, pipeline runner	RL evaluation and working alerts

Monthly milestones (README M1–M4)

Month	Goal	Done when
1	Data contract (IDs, zones, record schema, target outcome with the vet); install cameras; film 2–3 weeks including nights; register cows with photos; start labelling.	**M1**: labelled, versioned sample data and a documented schema
2	Fine-tune M1; first M4; records database; alert dispatcher; Analyst features designed.	Detector beats the COCO baseline on farm frames
3	M2 tuned; M3 trained; M6 security; A1 rules; simulator built. (Midterm buffer.)	**M2**: vision models evaluated on a held-out set
4	M5, A2 and A3, R1 and R2 trained; dashboard views complete.	Every model has a first result
5	Integration: clip to record to flag to priority to alert to feedback; run the pipeline daily on new footage.	**M3**: end-to-end demo with a real alert on all channels

6	Final evaluation, RL versus fixed-policy report, thesis writing, defence preparation. (Finals buffer.)	**M4**: RL evaluated offline; thesis submitted

9 Risks and what to cut if you fall behind

Risk	Plan
Labelling takes longer than expected	Use model-assisted labelling (let YOLO pre-label, then correct); label only what each model needs; share the work in month 1.
Cow ID is unreliable for look-alike or black cows	Store those observations as unassigned; restrict the evaluation to cows the ID model can tell apart; present ear-tag reading as future work.
Very few real illness cases	Present A1 and A2 as the system; train A3 with vet-reviewed flags and injected anomalies and state the limits; RL already uses a simulator.
Exams or course load cause delays	Two buffers in the plan; should-have items are dropped first.
Integration problems at the end	Fix the JSON and database contracts in month 1; run a weekly "thin end-to-end" with fake data from month 2.

Cut order if late: (1) M5 mouth-open → (2) A3 time-window head (keep a fixed 24–72 h statement) → (3) WhatsApp (keep SMS and dashboard) → (4) DQN comparison. Everything marked Must stays.

10 Evaluation

Model	Metric	Compared against
M1 detector	mAP50, recall in crowded and night scenes	COCO-pretrained YOLO11 (current)
M2 tracker	ID switches and IDF1 on 3–5 annotated clips	Default tracker settings
M3 cow ID	Top-1 accuracy, Recall@5, share of observations assigned	Random and colour-histogram baselines
M4, M5	Per-class precision and recall; minutes-per-day error against manual annotation	Rules only (no posture model)
M6 security	Precision and recall on staged test clips	None
A1, A2, A3	Recall, false alerts per 100 cow-days, lead time	Each other
R1, R2	Cases caught, mean lead time, false alarms, vet calls per episode	R1 fixed policy

Split data by cow and by recording period, never by random frames, so that near-duplicate frames do not inflate the results (README task 3).

11 Model summary

Agent	Model	Technique	Type	Priority	Owner
CV	M1 Cow detector	YOLO11s/m	Fine-tuned	Must	S1
	M2 Tracker	BoT-SORT + re-link	Algorithm	Must	S1
	M3 Cow ID	Siamese ResNet-50, triplet loss	Trained	Must	S1
	M4 Behaviour	YOLO11s-cls posture + zone rules	Trained + rules	Must	S2
	M5 Visible state	YOLO11n-cls or MobileNetV3	Trained	Should	S2
	M6 Security	YOLO11 COCO + zone and time rules	Pretrained	Must	S2
Analyst	A1 Baseline rules	Robust z-score	Rules	Must	S3
	A2 Anomaly detector	Isolation Forest	Unsupervised	Must	S3
	A3 Suspicion classifier	LightGBM + window head	Supervised	Should	S3
RL	R1 Fixed policy	Thresholds	Rules	Must	S4
	R2 PPO policy	Stable-Baselines3 PPO in a Gymnasium simulator	RL	Must	S4

Based on the project README, the project proposal and the existing code in the project folder. Hour and label counts are planning estimates; adjust them after month 1.